

MSDS 535 Week 5

Chapter 5

Overview and Objectives

- Compile Mining various kinds of patterns.
- Compare and contrast Mining compressed or approximate patterns.
- Understand constraint-based pattern mining.
- Interpret mining sequential patterns.
- Mining subgraph patterns.
- Compare and Contrast pattern mining: application examples.
- Summarize patter mining and the advanced methods.

Pattern Mining Methods Overview

- Mining various kinds of patterns
- Multilevel patterns
- Multidimensional patterns
- Continuous data patterns
- Rare patterns
- Negative patterns
- High-dimensional data patterns
- Compressed and approximate patterns
- Constraint utilization for cost reduction
- Sequential pattern mining
- Subgraph pattern mining
- Extending pattern mining for diverse applications

Pattern Mining Types

Frequent
pattern
mining

Rare pattern
mining

Negative
pattern
mining

Mining Various Kinds of Patterns

- Complex patterns are sought in massive data:
- Multilevel associations
- Multidimensional associations
- Quantitative association rules
- Patterns in high-dimensional space
- Rare patterns and negative patterns

Mining Multilevel Associations

Multilevel association rules involve patterns at different abstraction levels.

Efficient mining using concept hierarchies and support-confidence framework.

Top-down strategy for discovering frequent itemsets.

Mining Multilevel Associations

- Variations include uniform support, reduced support, and group-based support.
- Considerations for setting minimum support thresholds at different levels.
- Dealing with redundancy in multilevel association rules.
- Rules are considered redundant if support and confidence are similar to ancestor rules.

Single-Dimensional
vs.
Multidimensional
Associations

Association rules typically focus on a single predicate, e.g., "buys."

Multidimensional databases introduce the concept of dimensions.

Single-dimensional rules are common in transactional data.

Multidimensional Databases

- Sales and relational data are often integrated into data warehouses.
- Data warehouses store multidimensional data.
- Attributes such as item description, customer details, and more are recorded.
- Each attribute or dimension can be considered a predicate.

Multidimensional Association Rules



Multidimensional association rules contain multiple predicates.



Example: age, occupation, and buys in a single rule.



Rules can have repeated or non-repeated predicates.



Repeated predicates lead to hybrid-dimensional association rules.

Handling Quantitative Attributes

- Two approaches for quantitative attributes:
 1. Static discretization using predefined concept hierarchies.
 2. Dynamic discretization or clustering based on data distribution.
- Dynamic approach leads to (dynamic) quantitative association rules.

Searching for Predicate Sets

In multidimensional association rule mining, we search for frequent predicate sets.

A k -predicate set contains k conjunctive predicates.

Example: {age, occupation, buys} in a 3-predicate set.

Quantitative Attributes

- Relational and data warehouse data often involve quantitative attributes.
- These attributes can be discretized or treated as numeric values.
- Methods to mine quantitative association rules:
 1. Data cube-based mining.
 2. Clustering-based mining.
 3. Statistical analysis to uncover exceptional behaviors.

Data Cube-Based Mining

- Data cubes are suitable for mining multidimensional association rules.
- Data cubes store aggregates in multidimensional space.
- Support and confidence calculations for rules.
- Existing data cubes can be utilized if available.
- If not, on-the-fly creation may be necessary (iceberg cube computation).

Clustering- Based Mining



Quantitative association rules can be generated by clustering data.



Clusters represent dense regions of quantitative attributes.



Top-down and bottom-up approaches for clustering-based mining.



Challenges in high-dimensional data for bottom-up clustering.

Statistical Analysis

- Discover quantitative association rules indicating exceptional behavior.
- Define exceptional patterns based on statistical theory.
- Example: Identifying significant differences in mean values.
- Requires statistical tests to confirm rule validity.

Challenges in High-Dimensional Data

- Mining high-dimensional data poses challenges.
- Traditional methods may suffer from exponential search space growth.
- Two approaches to handle high-dimensional data:
 1. Vertical data format and pattern growth.
 2. Pattern-Fusion for identifying colossal patterns.

Pattern-Fusion for High-Dimensional Data

- Pattern-Fusion is an approach to mining colossal patterns.
- Colossal patterns are long and meaningful patterns.
- Traditional methods are challenged in finding colossal patterns.
- Pattern-Fusion takes leaps in the pattern search space.

Mining Rare Patterns

Rare patterns have support below a user-specified threshold.

Constraints can specify rarity conditions.

Mining rare patterns can be tackled with item- or group-based thresholds.

Mining Negative Patterns

- Negative patterns reflect negative correlations between items.
- Various definitions of negative patterns:
 - Frequency-based (not null-invariant).
 - Kulczynski measure-based (not null-invariant).
 - Statistical analysis-based (null-invariant).
- Example of negative patterns: Itemsets that rarely occur together.

Challenges in Frequent Pattern Mining

- The challenge of dealing with a large number of discovered patterns.
- Using a minimum support threshold to control pattern generation has limitations.
- Too low support leads to an excessive number of patterns, while too high support may discover only common patterns.

Mining Compressed or Approximate Patterns

- To reduce the set of frequent patterns while maintaining quality, compressed or approximate patterns can be mined.
- Top-k most frequent patterns concentrate on a specific set but may not epitomize representative patterns.
- Constraint-based mining and measures of interestingness can filter out uninteresting patterns.

Advanced Compression of Frequent Patterns



Two advanced compression methods building on closed and max-pattern concepts.



Clustering-based compression groups patterns by similarity and frequency support.



Summarization approach derives redundancy-aware top-k representative patterns.

Mining Compressed Patterns by Pattern Clustering

- Clustering techniques group similar objects; in this case, frequent patterns.
- Clusters use the δ -cluster tightness measure and select a representative pattern for each cluster.
- A pattern can belong to multiple clusters, and δ measures the tightness of a cluster.

Pattern Distance Measure



Pattern distance is defined using the support information of patterns.



It simplifies distance calculations by considering only the supports of the patterns.



The minimum support for a representative pattern is defined as min_supr .

Pattern Compression Problem

- Define a set of representative patterns R that covers each frequent pattern with respect to min_sup and min_supr .
- Finding a minimum set of representative patterns is NP-Hard but can be efficiently approximated.
 - nondeterministic polynomial-time hard
 - an NP-Hard problem is one for which a proposed solution can be checked in polynomial time but finding the solution itself may not be as straightforward.

Extracting Redundancy-Aware Top-k Patterns

- Reducing Patterns by Focusing on Significance and Redundancy:
 - Traditional top-k patterns select significant patterns, while k-summarized patterns focus on nonredundancy.
 - Redundancy-aware top-k patterns strike a balance between significance and redundancy.

Defining Significance and Redundancy

- Significance measures (objective or subjective) evaluate pattern interestingness.
- Redundancy between patterns is defined as $R(p, q)$.
- Approximating redundancy using distance measures.
- Redundancy-Aware Top-k Patterns:
 - Selecting patterns that are both significant and have low redundancy.
 - Optimizing the marginal significance to find high-significance, low-redundancy patterns.

Constraint-Based Pattern Mining

- Users specify constraints to guide the mining process.
- Constraints can be related to knowledge type, data, dimensions, interestingness, or pattern rules.
- Rule/Pattern Constraints:
 - Users may specify syntactic forms of rules or metarules to express expectations.
 - Mining system generates concrete rules matching the given metarules.

Pruning Strategies

Pruning pattern search space eliminates patterns that will be useless.

Pruning data search space identifies data objects that won't contribute to satisfiable patterns.

Pruning pattern space with pattern pruning constraints

- Constraints in pattern mining categorized into four types: antimonotonic, monotonic, convertible, and nonconvertible.
- Antimonotonic constraints prune patterns if they violate the constraint, and adding more items won't help satisfy it.
- Monotonic constraints, on the other hand, prune patterns if they satisfy the constraint because further expansion won't affect the constraint.
- Convertible constraints can change from antimonotonic to monotonic by rearranging the order of items.
- Some constraints are not antimonotonic or monotonic, making them harder to handle.

Pruning data space with data pruning constraints

- Data pruning constraints help prune data entries that won't contribute to satisfying patterns.
- Constraints like " $\text{sum}(\text{I.price}) \geq \100 " can prune data entries that can't meet the constraint.
- Data antimonotonicity can efficiently prune data entries if they don't satisfy the constraint.

Mining space pruning with succinctness constraints

Succinct constraints allow for direct pruning of data objects or pattern generation.

Constraints can be both data and pattern succinct, and they simplify the mining process.

Some constraints may belong to multiple categories, and their order may affect pruning effectiveness.

Sequential pattern mining: concepts and primitives

- Sequential pattern mining involves finding frequently occurring ordered events or subsequences.
- Sequences consist of ordered events, and events contain sets of items.
- Sequences have support, and frequent sequences are called sequential patterns.
- Sequences can be categorized by length, and subsequence relationships exist.
- The Apriori property applies to sequential patterns.

Scalable methods for mining sequential patterns

- Sequential pattern mining can be divided into methods for mining the full set and mining closed sequential patterns.
- Methods for mining the full set include GSP, SPADE, and PrefixSpan.
- GSP uses candidate generation and test, starting with frequent items and expanding sequentially.
- Candidates are generated using self-join and prune using the Apriori property.
- Efficient mining methods are needed due to the potentially large number of candidates.

Vertical Data Format and Apriori-like Approach

- Sequences can be mapped into vertical data format, represented as $\langle \text{itemset} : (\text{sequence_ID}, \text{event_ID}) \rangle$.
- Each itemset is associated with a sequence identifier and event identifier.
- Itemsets can occur in multiple sequences, forming ID_lists of itemsets.
- Support of k-sequence can be determined by joining ID_lists of its (k-1)-length subsequences.

SPADE Algorithm

- SPADE (Sequential Pattern Discovery using Equivalent Classes) is an Apriori-based sequential pattern mining algorithm.
- It uses vertical data format and requires one scan to find frequent 1-sequences.
- Candidate 2-sequences are formed by joining frequent single items with the same sequence identifier and sequential ordering of event identifiers.
- Itemset length is grown iteratively, and the process stops when no more frequent sequences can be found.

PrefixSpan Algorithm

- PrefixSpan is a pattern growth method that avoids candidate generation.
- It uses FP-trees to generate projected databases for frequent single items.
- These databases are mined recursively, avoiding candidate generation.
- Pseudo-projection is used to optimize performance when projected databases fit in memory.

Performance Comparison

PrefixSpan outperforms GSP and SPADE in overall performance.

Generating large candidate sets can consume significant memory and slow down candidate generate-and-test algorithms.

Closed sequential pattern mining can partially solve this problem.

Mining Closed Sequential Patterns

- Mining closed sequential patterns involves finding patterns containing no supersequence with the same support.
- It generates fewer sequences than mining the complete set of sequential patterns.
- Closed subsequences have the same expressive power as the full set of subsequences and are quicker to find.

Mining Multidimensional, Multilevel Sequential Patterns



Sequential pattern mining can consider additional information associated with sequence IDs and item IDs.



Mining multidimensional, multilevel sequential patterns explores patterns in a broader dimensional space.

Constraint-Based Mining of Sequential Patterns

- Constraint-based mining uses user-specified constraints to reduce the search space and find patterns of interest.
- Constraints can be related to duration, pattern length, event folding window, time gap between events, or pattern templates.

Examples of Constraints

- Duration constraints limit data selection within a specified period.
- Maximal or minimal length constraints restrict pattern length.
- Event folding window combines events occurring within a specified period.
- Time gap constraints specify time gaps between events in patterns.
- Pattern templates in the form of regular expressions define desired patterns (serial or parallel episodes).

Introduction to Graph Mining

Graphs are important for modeling complex structures in various domains, such as circuits, images, workflows, chemical compounds, and social networks.

Graph mining has become crucial in data mining due to the increasing demand for analyzing large structured datasets.

Frequent Subgraph Patterns

- Frequent substructures or subgraphs are fundamental patterns that can be discovered in collections of graphs.
- They serve various purposes, including characterizing graph sets, classifying graphs, and facilitating similarity search in graph databases.
- Recent studies have developed methods to mine frequent subgraphs and applied them in diverse applications.

Methods for Mining Frequent Subgraphs

- Apriori-based Approach:
- Apriori-based algorithms share similarities with frequent itemset mining and involve generating candidates and checking their frequency.
- They use a level-wise mining approach, starting with small-sized graphs and incrementally generating larger candidates.
- AprioriGraph is an algorithm that follows this approach.

Methods for Mining Frequent Subgraphs

- Pattern-growth Approach:
- Pattern-growth algorithms adopt a more flexible search method, including breadth-first search (BFS) and depth-first search (DFS).
- They aim to reduce the generation of duplicate graphs.
- The gSpan algorithm is an example of a pattern-growth approach that efficiently discovers frequent subgraphs.
- It uses a DFS lexicographic order to eliminate duplicate graphs and extends patterns through right-most extensions.
- gSpan can be implemented as either a depth-first search or a breadth-first search version.

DFS Codes and Mining Strategy

- DFS codes are used to represent graphs and their orderings.
- Edge order and sequence order are defined to build an order among edge sequences and edge labels, which is called DFS lexicographic order.
- The minimum DFS code of a graph is obtained through right-most extensions, ensuring the completeness of mining results.
- Duplicate graphs are eliminated by considering the minimum DFS code, reducing computational overhead.

gSpan Algorithm



The gSpan algorithm uses a depth-first search strategy and efficiently discovers frequent subgraphs.



It extends graph patterns while ensuring that their frequent descendants are found.



Nonminimum DFS codes are pruned, significantly improving efficiency.



A breadth-first search version of gSpan is also possible and yields similar performance to the depth-first search version while consuming less memory.

Mining Closed Frequent Substructures

- Closed frequent substructures address the issue of an explosive number of patterns generated by frequent subgraph mining.
- Closed graphs are those without proper supergraphs with the same support, and maximal subgraph patterns have no frequent superpatterns.
- Closed subgraph patterns provide a complete but more compact representation of frequent patterns.
- Efficient methods like CloseGraph exist for mining closed frequent graphs.

Extension of Pattern-Growth Approach

- Pattern-growth algorithms like gSpan can be extended for mining unlabeled, partially labeled, nonsimple, and directed graphs.
- The method can also be extended to mining disconnected graphs and frequent subtrees.
- Tree structures can be considered degenerated graphs for mining frequent subtrees.

Mining Substructure Patterns with User- Specified Constraints



Constraint-based mining of frequent substructures can be developed systematically.



Various constraints include element, set, or subgraph containment, geometric constraints, and value-sum constraints.



Handling multiple constraints and developing customized algorithms for specific cases may be necessary.

Mining Approximate Frequent Substructures

- Mining approximate frequent substructures allows for slight structural variations, reducing the number of patterns generated.
- SUBDUE, a system based on the Minimum Description Length principle, mines approximate frequent substructures.
- Research is ongoing to explore different techniques for mining approximate substructure patterns.

Mining Coherent Substructures

Coherent substructures are subgraphs with mutual information above a threshold.

Mining coherent substructures can efficiently prune redundant patterns.

A method for mining coherent substructures has been developed, particularly useful in applications like protein structure analysis.

Pattern Mining Application Examples

- Pattern mining is essential in various applications, including phrase mining in massive text data and software bug analysis.
- Phrase mining aims to extract meaningful phrases from unstructured text, enhancing data manipulation and named entity recognition.
- Quality criteria for phrases include popularity, concordance, informativeness, and completeness.

Phrase Quality Definition

- Phrase quality assesses how well a group of words make sense together based on four criteria.
- We represent phrase quality as a probability, $Q(v)$, ranging from 0 to 1. This probability indicates how likely the words in a sequence form a coherent phrase.
- Single words are inherently coherent, so their quality is set to 1. For longer phrases, quality needs to be learned from data, distinguishing good ($Q \approx 1$) from less coherent phrases ($Q \approx 0$).

Concordance Computation



Concordance evaluates how often words within a phrase co-occur.



Metrics like pointwise mutual information (PMI) measure concordance. Higher PMI values indicate stronger co-occurrence, suggesting higher quality.



Concordance features consider different ways to split a phrase into subunits, revealing how well its parts work together.

Informativeness Computation

- Informativeness gauges the usefulness of a phrase.
- Examining stopwords' positions in a phrase helps; if they're at the start or end, the phrase might be less informative.
- Computing average inverse document frequency (IDF) reveals the importance of words within a phrase. High-quality phrases have reasonable average IDF.

Phrasal Segmentation

- Phrasal segmentation involves dividing a sequence of words into smaller parts, either single words or phrases.
- The segmentation process helps extract quality phrases from text.
- Segmentation can be ambiguous, but it doesn't need to be perfect. Even with some errors, a reasonably accurate segmentation retains support for quality phrases.

Phrase Mining Methods

- Three approaches for mining quality phrases: Unsupervised, Weakly Supervised, and Distantly Supervised.
- Unsupervised method (ToPMine) discovers phrases purely from text statistics, based on frequent phrase patterns.
- Weakly Supervised method (SegPhrase) uses a small set of labeled quality phrases to rank and extract phrases from text.
- Distantly Supervised method (AutoPhrase) leverages external knowledge bases to identify quality phrases, minimizing manual labeling.

Mining Copy-Paste Bugs in Software Programs

- Pattern mining applied to software code helps uncover copy-paste bugs.
- Copy-pasting is common in coding but can introduce errors if changes aren't consistent.
- Transforming code into sequences of numbers allows finding copied code blocks, even with inserted statements.
- Detecting inconsistencies between pasted code blocks can identify copy-paste bugs automatically.

Summary

- Frequent pattern mining research extends beyond basic concepts, encompassing various patterns, mining methods, and applications.
- Advanced patterns like multilevel associations, multidimensional associations, quantitative association rules, rare patterns, and negative patterns can be mined.
- Multilevel associations involve multiple abstraction levels, while multidimensional associations contain multiple dimensions.
- Rare patterns and negative patterns, which occur infrequently, can highlight exceptional data behavior.

Summary

- Constraint-based mining helps direct the process based on user-defined constraints, including pattern-pruning and data-pruning constraints.
- Methods exist for mining patterns in high-dimensional space, including pattern growth and Pattern-Fusion approaches.
- Compressed patterns and approximate patterns can be mined to reduce the pattern count.
- Sequential pattern mining involves ordered event or subsequence mining with efficient algorithms like GSP and PrefixSpan.

Summary

- Subgraph pattern mining focuses on frequent subgraphs in a collection of graphs, with algorithms like gSpan and CloseGraph.
- Pattern mining finds applications in market analysis, phrase mining from text, and software copy-paste bug detection.
- Methods like ToPMine, SegPhrase, AutoPhrase, and CP-Miner are introduced for phrase and bug mining.